

ROS2 Day1 과제 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-09

목차

1. Task01 – 터미널에서 토픽 발행을 통해 삼각형, 원, 사각형 그리기

- 1.1 과제 개요
- 1.2 구현 방법
- 1.3 실행 결과
- 1.4 느낀 점

2. Task02 – Topic 통신 구현

- 2.1 과제 개요
- 2.2 C++ / Python Publisher, Subscriber 구현
- 2.3 Topic 통신 및 rqt_graph
- 2.4 실행 결과
- 2.5 느낀 점

3. Task03 – turtlesim 패키지를 활용해 그림 그리기

- 3.1 과제 개요
- 3.2 프로그램 구현
- 3.3 실행 준비 및 작동 영상
- 3.4 느낀 점

1. Task01 – 터미널에서 토픽 발행을 통해 삼각형, 원, 사각형 그리기

1.1 과제 개요

Task01에서는 ROS2의 turtlesim을 실행한 후 터미널에서 직접 토픽을 발행하여 거북이를 움직이고, 사각형, 삼각형, 원을 각각 그려보았다. 실습을 진행하기 전에 다른 ROS2 통신과 겹치지 않도록 ROS_DOMAIN_ID를 6으로 설정하였다.

```
export ROS_DOMAIN_ID=6
```

이후 turtlesim을 다음 명령어로 실행하였다.

```
ros2 run turtlesim turtlesim_node
```

거북이의 이동은 /turtle1/cmd_vel 토픽에 geometry_msgs/msg/Twist 메시지를 발행하는 방식으로 제어하였다. linear.x 값을 이용하여 직선 이동을 제어하고, angular.z 값을 이용하여 회전하도록 하였다.

1.2 구현 방법

사각형

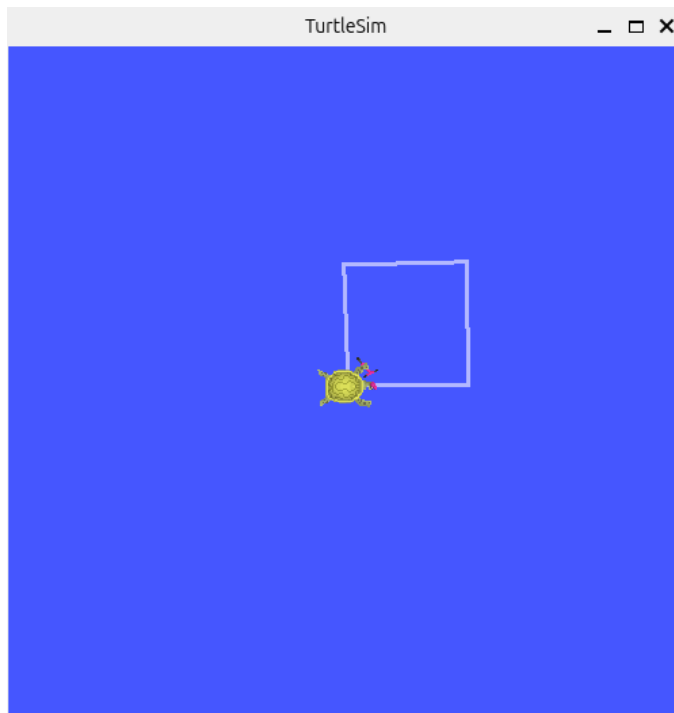
사각형은 일정 시간 동안 직진한 뒤 약 90도 회전하는 동작을 반복하는 방식으로 구현하였다. 직진 시에는 다음 명령어를 사용하였다.

```
timeout 2s ros2 topic pub -r 10 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 1.0},
 angular: {z: 0.0}}"
```

회전 시에는 다음 명령어를 사용하였다.

직진과 회전을 반복하여 네 개의 변을 만들었으며, 실행 결과 사각형 형태가 만들어지는 것을 확인하였다.

```
timeout 1s ros2 topic pub -r 10 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 0.0},
 angular: {z: 1.57}}"
```



삼각형

삼각형도 직선 이동과 회전을 반복하는 방식으로 구현하였다. 한 변을 이동한 뒤 다음 변을 향하도록 약 120도 회전하도록 설정하였다.

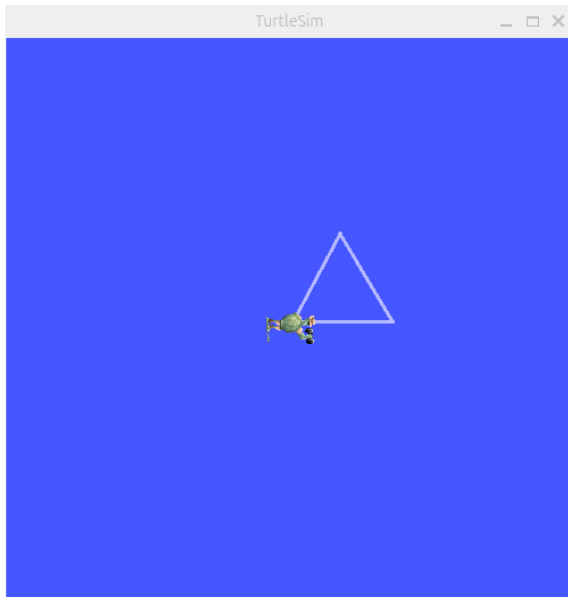
직진 명령은 다음과 같다.

```
timeout 2s ros2 topic pub -r 10 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 1.0},
  angular: {z: 0.0}}"
```

회전 명령은 다음과 같다.

```
timeout 1s ros2 topic pub -r 10 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 0.0},
  angular: {z: 2.094}}"
```

직진과 회전을 세 번 반복하여 삼각형을 완성하였다.

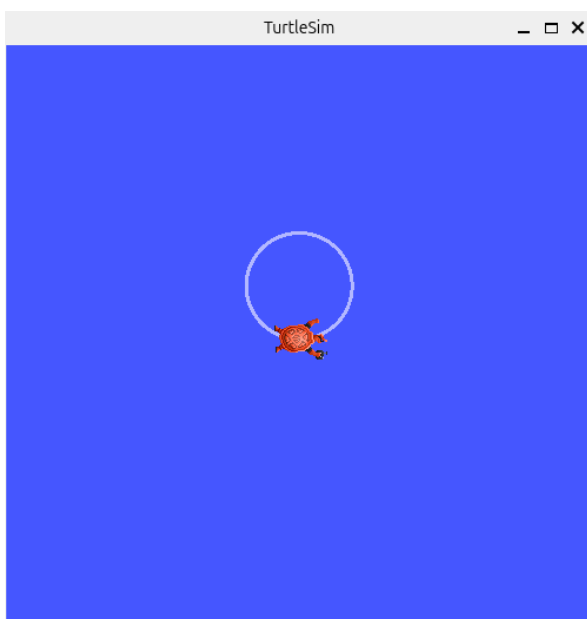


원

원은 사각형과 삼각형처럼 직진과 회전을 따로 수행하지 않고, 선속도와 각속도를 동시에 적용하는 방식으로 구현하였다.

```
timeout 6.3s ros2 topic pub -r 10 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 1.0},
 angular: {z: 1.0}}"
```

linear.x와 angular.z에 동시에 값을 주어 거북이가 앞으로 이동하면서 계속 방향을 바꾸도록 하였다. 이를 통해 원 형태의 궤적을 만들 수 있었다.



1.3 실행 결과

터미널에서 `/turtle1/cmd_vel` 토픽에 직접 메시지를 발행하여 사각형, 삼각형, 원을 모두 구현할 수 있었다. 사각형과 삼각형은 직선 이동과 회전을 반복하는 방식으로 구현하였으며, 회전 정도를 다르게 설정하여 각각 다른 형태를 만들었다. 원의 경우에는 선속도와 각속도를 동시에 적용하여 연속적으로 회전하면서 이동하도록 하였다.

1.4 느낀 점

이번 과제를 진행하면서 ROS2에서 토픽을 이용하여 거북이의 이동을 직접 제어하는 방법을 알 수 있었다. 처음에는 `linear.x`와 `angular.z` 값이 실제 움직임에 어떤 영향을 주는지 정확히 알기 어려웠지만, 값을 직접 변경하면서 직진 속도와 회전 속도의 차이를 확인할 수 있었다.

또한 사각형과 삼각형은 직진과 회전을 반복하여 만들 수 있었고, 원은 직진과 회전을 동시에 적용해야 한다는 차이를 확인하였다. 이를 통해 같은 Twist 메시지를 사용하더라도 선속도와 각속도를 어떻게 조합하느냐에 따라 거북이의 움직임이 달라진다는 것을 이해할 수 있었다.

2. Task02 – Topic 통신 구현

2.1 과제 개요

Task02에서는 C++ 패키지와 Python 패키지를 각각 생성하고, 각 패키지 안에 Publisher 노드와 Subscriber 노드를 작성하여 Topic 통신을 구현하였다.

통신에 사용한 메시지는 int, string, float, bool 형태로 구성하였으며, Subscriber에서는 전달받은 데이터를 callback 함수를 통해 터미널에 출력하도록 하였다.

또한 같은 패키지 내부의 Publisher와 Subscriber 간 통신뿐만 아니라 C++ 패키지와 Python 패키지 사이의 통신도 확인하였다. 마지막으로 rqt_graph를 이용하여 노드와 토픽이 실제로 어떻게 연결되어 있는지 확인하였다.

2.2 C++ / Python Publisher, Subscriber 구현

C++ 패키지

C++ 패키지는 Publisher와 Subscriber를 각각 하나의 노드로 작성하였다. 소스 파일과 헤더 파일을 분리하여 다음과 같이 구성하였다.

```
changgyu_task2_cpp
├─ include/changgyu_task2_cpp
│  ├─ publisher.hpp
│  └─ subscriber.hpp
├─ src
│  ├─ publisher.cpp
│  └─ subscriber.cpp
├─ CMakeLists.txt
└─ package.xml
```

Publisher에서는 Int32, String, Float32, Bool 타입의 Publisher를 생성하였다.

```
int_publisher_ =
    this->create_publisher<std_msgs::msg::Int32>(
        "/task2/int_data", 10);

string_publisher_ =
    this->create_publisher<std_msgs::msg::String>(
        "/task2/string_data", 10);
```

같은 방식으로 Float32, Bool Publisher도 생성하였다.

1초마다 callback 함수가 실행되도록 timer를 설정하였으며, callback 내부에서 각 메시지에 값을 저장한 뒤 publish하도록 작성하였다.

```
int_msg.data = count_;
string_msg.data = "Hello ROS2 C++";
float_msg.data = 3.14f;
bool_msg.data = (count_ % 2 == 0);
```

Subscriber에서는 Publisher와 동일한 토픽을 구독하도록 설정하고, 각각의 callback 함수에서 수신한 값을 터미널에 출력하였다.

```
void Task2Subscriber::int_callback(
    const std_msgs::msg::Int32::SharedPtr msg)
{
    RCLCPP_INFO(
        this->get_logger(),
        "Received int: %d",
        msg->data);
}
```

Python 패키지

Python 패키지는 다음과 같이 Publisher와 Subscriber 파일을 각각 작성하였다.

```
changgyu_task2_py
├─ changgyu_task2_py
│   ├─ publisher.py
│   ├─ subscriber.py
│   └─ __init__.py
├─ setup.py
├─ setup.cfg
└─ package.xml
```

Python Publisher에서도 C++과 동일하게 Int32, String, Float32, Bool 메시지를 사용하였다.

```
self.int_publisher = self.create_publisher(
    Int32, '/task2/int_data', 10
)

self.string_publisher = self.create_publisher(
    String, '/task2/string_data', 10
)
```

timer callback에서 각 데이터를 생성하여 publish하였고, bool 값은 count 값에 따라 True와 False가 번갈아 나오도록 설정하였다.

```
bool_msg.data = (self.count % 2 == 0)
```


Python Subscriber에서는 각각의 토픽을 구독하고 callback을 통해 전달받은 값을 출력하도록 작성하였다.

2.3 Topic 통신 및 rqt_graph

먼저 하나의 패키지 내부에서 Publisher와 Subscriber 간의 통신을 확인하였다.

C++ 패키지 내부 통신

C++ Publisher 노드를 실행한 뒤 별도의 터미널에서 C++ Subscriber 노드를 실행하였다.

Publisher에서는 int, string, float, bool 값을 계속 발행하였고, Subscriber에서는 동일한 데이터가 정상적으로 출력되는 것을 확인하였다.

```
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870056.502024334] [task2_cpp_publisher]: Publish -> int: 15, string:
Hello ROS2 C++, float: 3.14, bool: false
[INFO] [1788870057.502030547] [task2_cpp_publisher]: Publish -> int: 16, string:
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870058.502012514] [task2_cpp_publisher]: Publish -> int: 17, string:
Hello ROS2 C++, float: 3.14, bool: false
[INFO] [1788870059.502016483] [task2_cpp_publisher]: Publish -> int: 18, string:
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870060.502075349] [task2_cpp_publisher]: Publish -> int: 19, string:
Hello ROS2 C++, float: 3.14, bool: false
[INFO] [1788870061.501949377] [task2_cpp_publisher]: Publish -> int: 20, string:
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870062.501953779] [task2_cpp_publisher]: Publish -> int: 21, string:
Hello ROS2 C++, float: 3.14, bool: false
[INFO] [1788870063.502000555] [task2_cpp_publisher]: Publish -> int: 22, string:
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870064.501994517] [task2_cpp_publisher]: Publish -> int: 23, string:
Hello ROS2 C++, float: 3.14, bool: false
[INFO] [1788870065.501935293] [task2_cpp_publisher]: Publish -> int: 24, string:
Hello ROS2 C++, float: 3.14, bool: true
[INFO] [1788870066.501986004] [task2_cpp_publisher]: Publish -> int: 25, string:
Hello ROS2 C++, float: 3.14, bool: false

[INFO] [1788870062.502496662] [task2_cpp_subscriber]: Received string: Hello ROS
2 C++
[INFO] [1788870062.502541272] [task2_cpp_subscriber]: Received int: 21
[INFO] [1788870063.502304747] [task2_cpp_subscriber]: Received bool: true
[INFO] [1788870063.502452015] [task2_cpp_subscriber]: Received float: 3.14
[INFO] [1788870063.502510404] [task2_cpp_subscriber]: Received string: Hello ROS
2 C++
[INFO] [1788870063.502554293] [task2_cpp_subscriber]: Received int: 22
[INFO] [1788870064.502299333] [task2_cpp_subscriber]: Received bool: false
[INFO] [1788870064.502426559] [task2_cpp_subscriber]: Received float: 3.14
[INFO] [1788870064.502493821] [task2_cpp_subscriber]: Received string: Hello ROS
2 C++
[INFO] [1788870064.502534741] [task2_cpp_subscriber]: Received int: 23
[INFO] [1788870065.502269925] [task2_cpp_subscriber]: Received bool: true
[INFO] [1788870065.502391070] [task2_cpp_subscriber]: Received float: 3.14
[INFO] [1788870065.502452528] [task2_cpp_subscriber]: Received string: Hello ROS
2 C++
[INFO] [1788870065.502492332] [task2_cpp_subscriber]: Received int: 24
[INFO] [1788870066.502257887] [task2_cpp_subscriber]: Received bool: false
[INFO] [1788870066.502372884] [task2_cpp_subscriber]: Received float: 3.14
[INFO] [1788870066.502417513] [task2_cpp_subscriber]: Received string: Hello ROS
2 C++
[INFO] [1788870066.502472586] [task2_cpp_subscriber]: Received int: 25
```

Python 패키지 내부 통신

Python에서도 같은 방식으로 Publisher와 Subscriber를 각각 실행하였다.

Publisher에서 Hello ROS2 Python, 3.14, 정수값, True/False 값을 발행하였으며 Subscriber가 동일한 값을 수신하는 것을 확인하였다.

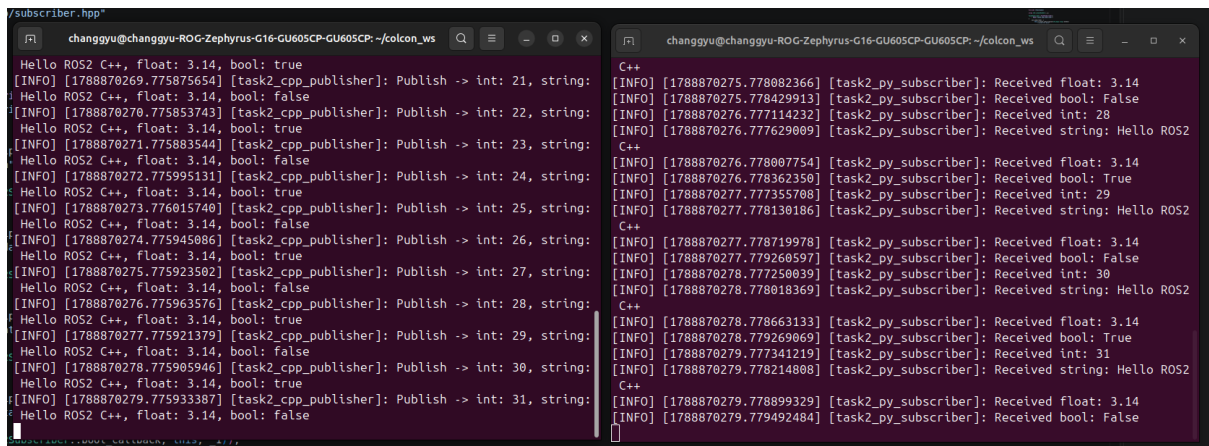
```
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869331.406452219] [task2_py_publisher]: Publish -> int: 43, string:
Hello ROS2 Python, float: 3.14, bool: False
[INFO] [1788869332.406474433] [task2_py_publisher]: Publish -> int: 44, string:
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869333.406538110] [task2_py_publisher]: Publish -> int: 45, string:
Hello ROS2 Python, float: 3.14, bool: False
[INFO] [1788869334.406497136] [task2_py_publisher]: Publish -> int: 46, string:
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869335.406369989] [task2_py_publisher]: Publish -> int: 47, string:
Hello ROS2 Python, float: 3.14, bool: False
[INFO] [1788869336.406430534] [task2_py_publisher]: Publish -> int: 48, string:
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869337.406412759] [task2_py_publisher]: Publish -> int: 49, string:
Hello ROS2 Python, float: 3.14, bool: False
[INFO] [1788869338.406403168] [task2_py_publisher]: Publish -> int: 50, string:
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869339.406474255] [task2_py_publisher]: Publish -> int: 51, string:
Hello ROS2 Python, float: 3.14, bool: False
[INFO] [1788869340.406372511] [task2_py_publisher]: Publish -> int: 52, string:
Hello ROS2 Python, float: 3.14, bool: True
[INFO] [1788869341.406263129] [task2_py_publisher]: Publish -> int: 53, string:
Hello ROS2 Python, float: 3.14, bool: False

Python
[INFO] [1788869337.407792276] [task2_py_subscriber]: Received float: 3.14
[INFO] [1788869337.408140653] [task2_py_subscriber]: Received bool: False
[INFO] [1788869338.406947124] [task2_py_subscriber]: Received int: 50
[INFO] [1788869338.407425860] [task2_py_subscriber]: Received string: Hello ROS2
Python
[INFO] [1788869338.407800505] [task2_py_subscriber]: Received float: 3.14
[INFO] [1788869338.408145302] [task2_py_subscriber]: Received bool: True
[INFO] [1788869339.407801393] [task2_py_subscriber]: Received int: 51
[INFO] [1788869339.407476063] [task2_py_subscriber]: Received string: Hello ROS2
Python
[INFO] [1788869339.407854586] [task2_py_subscriber]: Received float: 3.14
[INFO] [1788869339.408218863] [task2_py_subscriber]: Received bool: False
[INFO] [1788869340.406809112] [task2_py_subscriber]: Received int: 52
[INFO] [1788869340.407291867] [task2_py_subscriber]: Received string: Hello ROS2
Python
[INFO] [1788869340.407661251] [task2_py_subscriber]: Received float: 3.14
[INFO] [1788869340.408002003] [task2_py_subscriber]: Received bool: True
[INFO] [1788869341.406691717] [task2_py_subscriber]: Received int: 53
[INFO] [1788869341.407150946] [task2_py_subscriber]: Received string: Hello ROS2
Python
[INFO] [1788869341.407592170] [task2_py_subscriber]: Received float: 3.14
[INFO] [1788869341.407946932] [task2_py_subscriber]: Received bool: False
```

C++ Publisher → Python Subscriber

C++ Publisher와 Python Subscriber를 동시에 실행하여 서로 다른 패키지 사이에서도 같은 토픽을 이용하면 통신이 가능한지 확인하였다.

C++ Publisher에서 발행한 Hello ROS2 C++, 3.14, 정수값과 true/false 값이 Python Subscriber 터미널에서 정상적으로 출력되었다.

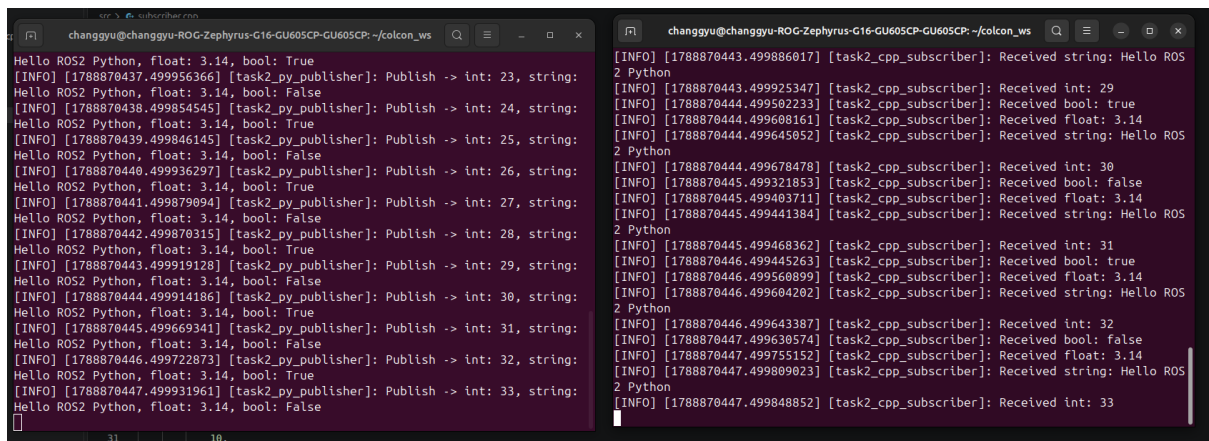


The image shows two terminal windows side-by-side. The left window, titled 'changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws', shows the output of a C++ Publisher. It prints 'Hello ROS2 C++, float: 3.14, bool: true' followed by a series of '[INFO] [timestamp] [task2_cpp_publisher]: Publish -> int: [value], string: Hello ROS2 C++, float: 3.14, bool: [true/false]'. The right window, titled 'changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws', shows the output of a Python Subscriber. It prints 'C++' followed by a series of '[INFO] [timestamp] [task2_py_subscriber]: Received float: 3.14', '[INFO] [timestamp] [task2_py_subscriber]: Received bool: False/True', and '[INFO] [timestamp] [task2_py_subscriber]: Received string: Hello ROS2 C++'.

Python Publisher → C++ Subscriber

반대 방향으로 Python Publisher와 C++ Subscriber를 실행하였다.

Python에서 발행한 Hello ROS2 Python, 3.14, 정수값, True/False를 C++ Subscriber에서 정상적으로 수신하는 것을 확인하였다.



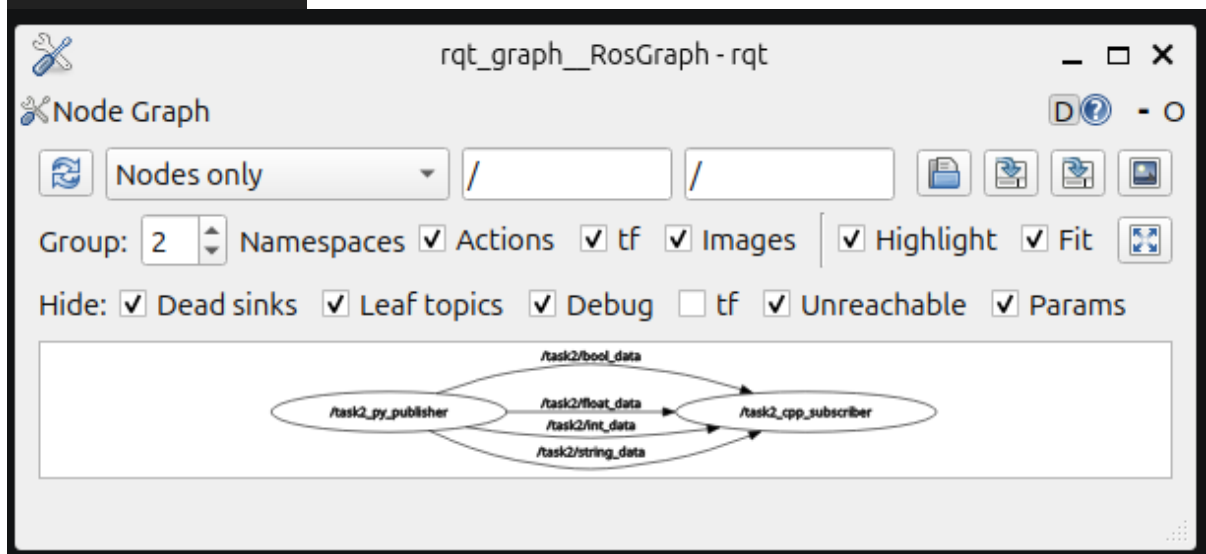
The image shows two terminal windows side-by-side. The left window, titled 'changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws', shows the output of a Python Publisher. It prints 'Hello ROS2 Python, float: 3.14, bool: True' followed by a series of '[INFO] [timestamp] [task2_py_publisher]: Publish -> int: [value], string: Hello ROS2 Python, float: 3.14, bool: [True/False]'. The right window, titled 'changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~/colcon_ws', shows the output of a C++ Subscriber. It prints '[INFO] [timestamp] [task2_cpp_subscriber]: Received string: Hello ROS2 Python' followed by a series of '[INFO] [timestamp] [task2_cpp_subscriber]: Received int: [value]', '[INFO] [timestamp] [task2_cpp_subscriber]: Received bool: true/false', and '[INFO] [timestamp] [task2_cpp_subscriber]: Received string: Hello ROS2 Python'.

rqt_graph 연결 구조 확인

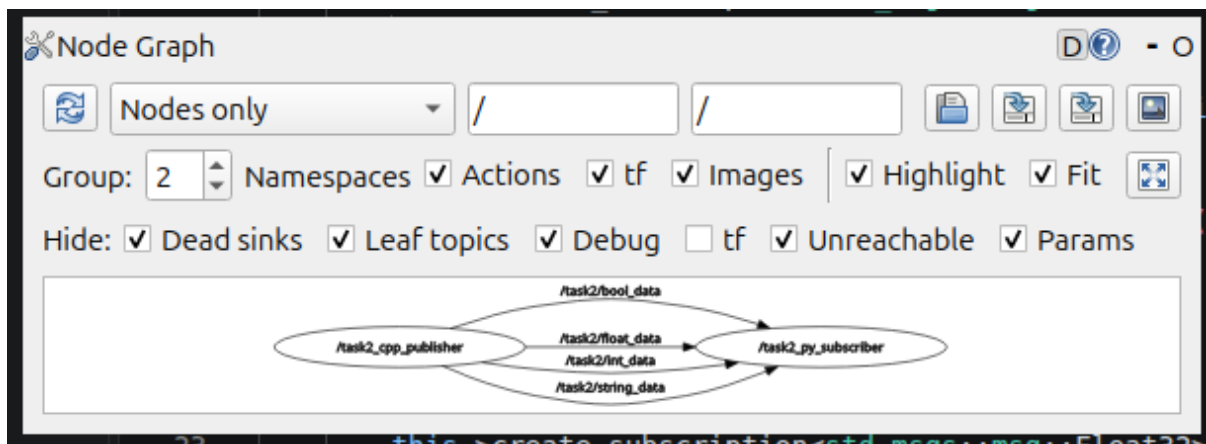
Topic 통신이 실제로 어떤 구조로 연결되어 있는지 확인하기 위해 rqt_graph를 실행하였다.

Python Publisher와 C++ Subscriber를 실행했을 때 다음 네 개의 토픽을 통해 두 노드가 연결되는 것을 확인하였다.

```
/task2/int_data  
/task2/string_data  
/task2/float_data  
/task2/bool_data
```



반대 방향에서도 C++ Publisher와 Python Subscriber 사이에 동일한 네 개의 토픽이 연결되는 것을 확인하였다.



2.4 실행 결과

C++과 Python에서 각각 Publisher와 Subscriber를 작성한 뒤 실행한 결과, 같은 패키지 내부의 노드 간 Topic 통신이 정상적으로 이루어지는 것을 확인하였다.

또한 C++과 Python이 서로 다른 언어로 작성되어 있어도 동일한 `std_msgs` 메시지 타입과 같은 토픽 이름을 사용하면 데이터를 주고받을 수 있다는 것을 확인하였다.

`int`, `string`, `float`, `bool` 메시지를 모두 정상적으로 송수신하였으며, Subscriber callback에서 각각의 데이터를 터미널에 출력할 수 있었다.

`rqt_graph`에서는 Publisher와 Subscriber 사이에 사용한 네 개의 토픽이 연결되어 있는 것을 직접 확인할 수 있었다.

2.5 느낀 점

이번 과제를 통해 ROS2의 Publisher와 Subscriber가 Topic을 이용하여 데이터를 주고받는 구조를 직접 구현해볼 수 있었다.

처음에는 C++과 Python이 서로 다른 언어이기 때문에 별도의 통신 방법이 필요할 것이라고 생각했지만, 같은 메시지 타입과 토픽을 사용하면 서로 통신할 수 있다는 점을 확인할 수 있었다.

또한 코드만 실행했을 때는 Publisher와 Subscriber 사이의 연결 관계가 잘 보이지 않았지만, `rqt_graph`를 사용하면서 어떤 노드가 어떤 토픽을 통해 연결되어 있는지 한눈에 확인할 수 있어서 Topic 통신 구조를 이해하는 데 도움이 되었다.

3. Task03 – turtlesim 패키지를 활용해 그림 그리기

3.1 과제 개요

Task03에서는 C++로 ROS2 패키지를 제작하고, 키보드 입력을 이용하여 turtlesim의 거북이를 제어하도록 구현하였다. W, A, S, D 키를 이용해 이동과 회전을 제어할 수 있도록 하였으며, 삼각형, 사각형, 원을 그릴 수 있도록 작성하였다.

또한 도형마다 선의 색상과 굵기가 구분되도록 설정하였다. 삼각형은 빨간색, 사각형은 초록색, 원은 파란색으로 설정하였고 각 도형의 굵기도 서로 다르게 적용하였다.

3.2 프로그램 구현

C++ 패키지는 헤더 파일과 소스 파일을 분리하여 다음과 같이 구성하였다.

```
changgyu_task3_cpp
├─ include/changgyu_task3_cpp
│   └─ task3.hpp
├─ src
│   └─ task3.cpp
├─ CMakeLists.txt
└─ package.xml
```

거북이 이동에는 /turtle1/cmd_vel 토픽을 사용하였으며, geometry_msgs/msg/Twist 메시지의 linear.x와 angular.z 값을 이용하여 직선 이동과 회전을 구현하였다.

이동에 필요한 동작은 move() 함수로 하나로 구성하였다.

```
void move(double linear,
          double angular,
          double time);
```

linear 값에는 직선 이동 속도, angular 값에는 회전 속도, time에는 동작 시간을 전달하였다. 하나의 함수를 이용하여 직진, 후진, 좌우 회전, 원 그리기를 모두 처리할 수 있도록 하였다.

키보드 입력은 getKey() 함수를 이용하여 Enter를 누르지 않고 한 글자씩 입력받도록 구현하였다.

```
char Task3::getKey()
{
    char key;
    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    read(STDIN_FILENO, &key, 1);
    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

    return key;
}
```

입력된 키는 run() 함수에서 판단하도록 하였다. W는 전진, S는 후진, A와 D는 좌우 회전에 사용하였다.

도형 선택에는 1, 2, 3 키를 추가로 사용하였다.

```
1 : Triangle Mode
2 : Square Mode
3 : Circle Mode

W : Forward
S : Backward
A : Turn Left
D : Turn Right
R : Reset
Q : Quit
```

삼각형 모드에서는 회전 시 약 120도 회전하도록 하였고, 사각형 모드에서는 약 90도 회전하도록 설정하였다. 원 모드에서는 선속도와 각속도를 동시에 적용하여 한 바퀴 회전하도록 구현하였다.

색상 및 굵기 설정

선의 색상과 굵기는 turtlesim의 set_pen 서비스를 이용하여 변경하였다.

```
void setPen(int r, int g,
            int b, int width);
```

각 도형은 다음과 같이 구분하였다.

```
삼각형 : 빨간색 / 굵기 2
사각형 : 초록색 / 굵기 6
원      : 파란색 / 굵기 10
```

3.3 실행 준비 및 작동 영상

프로그램 실행을 위해 먼저 turtlesim 노드를 실행하였다.

```
ros2 run turtlesim turtlesim_node
```

이후 별도의 터미널에서 Task03 패키지의 실행 파일을 실행하였다.

```
cd ~/colcon_ws
source install/setup.bash
ros2 run changgyu_task3_cpp task3_node
```

프로그램이 정상적으로 실행되면 터미널에 도형 선택과 키보드 조작 방법이 출력되도록 하였다.

```

1 : Triangle
2 : Square
3 : Circle
W/S : Forward/Backward
A/D : Left/Right
R : Reset
Q : Quit

```

```

changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: $ source ~/.bashrc
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: $ ros2 run turtlesim turtlesim
[INFO] [1788923347.527981638] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1788923347.536704541] [turtlesim]: Spawning turtle [turtle] at x=5.544445, y=5.544445, theta=0.868880

```



```

changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: ~
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: $ source ~/.bashrc
changgyu@changgyu-ROG-Zephyrus-G16-GU605CP-GU605CP: $ ros2 run turtlesim turtlesim
[INFO] [1788923347.527981638] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1788923347.536704541] [turtlesim]: Spawning turtle [turtle] at x=5.544445, y=5.544445, theta=0.868880

```



삼각형, 사각형, 원을 그리는 실제 동작 과정과 도형별 색상 및 굵기 변화는 작동 영상으로 별도 첨부하였다.

3.4 느낀 점

Task01에서는 터미널에서 직접 토픽을 발행하여 거북이를 움직였지만, 이번 과제에서는 C++ 프로그램을 작성하여 키보드 입력에 따라 거북이를 제어하였다. 직접 Publisher를 생성하고 Twist 메시지를 발행하면서 이전 과제에서 사용했던 토픽 통신을 프로그램 내부에서 어떻게 활용하는지 이해할 수 있었다.

처음에는 키를 누르는 시간을 직접 조절하여 회전 각도를 맞추려고 하여 도형을 일정하게 그리기 어려웠다. 이후 이동 시간과 회전 시간을 코드에서 정하도록 수정하면서 삼각형과 사각형을 더 일정하게 그릴 수 있었다. 또한 set_pen 서비스를 이용하여 단순히 거북이를 움직이는 것뿐만 아니라 선의 색상과 굵기도 변경할 수 있다는 것을 확인하였다.